

Informe Quipux Polucion - Prueba Tecnica

ALEXÁNDER JOSÉ PASSO POLANCO

August 2026

1 Introduction

En este trabajo, se desarrolla un algoritmo de ML para la predicción de la concentración del material particulado $PM_{2.5}$, usando información historica y variables meteorológicas. Inicialmente se realizó exploración y preparación del conjunto de datos. Luego el desarrollo, implementación y evaluación de dos modelos: **LSTM** y **XGBoost**. Finalmente se escoge el mejor modelo teniendo en cuenta métricas de error, capacidad de generalización, rendiendo y usabilidad.

2 Comprensión del conjunto de datos

En primer lugar se cuenta con dos fuentes de datos: **data.txt** que es la base de datos historicos que comprende los años 2010 – 2015, Y **dic_data.txt** que es un archivo donde se explica cada una de las columnas del dataset. Se realizó la revisión del conjunto de datos, para identificar las características y variable objetivo, y por último definir las transformaciones necesarias antes de aplicar los modelos de ML. El conjunto de datos contiene información temporal, mediciones relacionadas con las condiciones meteorológicas y registros de concentración de PM2.5. A partir del diccionario de datos se identificaron las variables disponibles, su tipo y su posible utilización dentro del modelo.

Table 1: Descripción de las variables utilizadas

Variable	Tipo	Descripción	Uso
date	Fecha	Fecha y hora	Feature temporal
pm2.5	Numérica	Concentración de PM2.5	Target
DEWP	Numérica	Punto de rocío	Feature
TEMP	Numérica	Temperatura	Feature
PRES	Numérica	Presión	Feature
cbwd	Categorica	Dirección del viento	Feature
Iws	Numérica	Velocidad acumulada del viento	Feature
Is	Numérica	Variable del dataset	Feature
Ir	Numérica	Variable del dataset	Feature

3 Exploración de los datos

Antes de aplicar algún modelo de ML, se realizó un análisis exploratorio de los datos (EDA) con el fin de identificar características relevantes y problemas que pudieran afectar el correcto entrenamiento, tales como valores nulos, entre otros. Inicialmente se verificó la estructura del dataset: cantidad de registros, variables y tipos de datos, calidad de los datos, y distribución de la variable objetivo. Este análisis detallado puede encontrarse en el notebook **01_eda_and_data_cleaning**

4 Preparación de los datos

Una vez finalizada la exploración, se realizó la preparación del conjunto de datos para adaptarlo a los requerimientos de los modelos seleccionados. Esta etapa comprende las transformaciones necesarias sobre las variables temporales y categoricas, la construcción de características históricas y la división del

conjunto de datos en entrenamiento, validación y prueba. Debido a la naturaleza temporal del problema, las transformaciones se realizaron procurando mantener el orden cronológico de las observaciones y evitar que información correspondiente a períodos futuros estuviera disponible durante el entrenamiento.

5 Modelos implementados de ML

Una vez realizada la exploración, limpieza y transformación de los datos, se procedió a la etapa de modelación con el objetivo de desarrollar un algoritmo capaz de predecir la concentración de PM2.5. Debido a la naturaleza temporal de los datos y a la existencia de una dependencia entre las mediciones actuales y los valores registrados previamente, se consideraron diferentes enfoques de modelación.

En primer lugar, se implementó una red neuronal recurrente de tipo Long Short-Term Memory (LSTM), debido a su capacidad para trabajar con secuencias y aprender relaciones temporales entre las observaciones. Posteriormente, se implementó el algoritmo XGBoost, utilizando las variables meteorológicas, la dirección del viento y las características históricas de PM2.5 construidas mediante variables *lag*.

Para comparar objetivamente el desempeño de los modelos se utilizaron cuatro métricas: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) y el coeficiente de determinación R^2 . Estas métricas permiten evaluar los errores de predicción desde diferentes perspectivas y determinar la capacidad de los modelos para reproducir los valores reales de concentración de PM2.5.

5.1 Métricas de evaluación

El **Mean Absolute Error (MAE)** representa el promedio de la diferencia absoluta entre los valores reales y los valores predichos. Esta métrica permite interpretar directamente cuánto se desvía, en promedio, la predicción respecto al valor real. Su expresión matemática es:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (1)$$

donde y_i corresponde al valor real, $\hat{y}_i$ al valor predicho por el modelo y n al número total de observaciones. Un valor de MAE más bajo representa un menor error promedio.

El **Mean Squared Error (MSE)** calcula el promedio de los errores al cuadrado. Debido a que los errores son elevados al cuadrado, esta métrica asigna un mayor peso a las predicciones que presentan errores de mayor magnitud:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2)$$

Por lo tanto, un MSE menor indica un menor error cuadrático promedio. Sin embargo, debido al proceso de elevar los errores al cuadrado, su interpretación directa resulta menos intuitiva que la del MAE.

El **Root Mean Squared Error (RMSE)** corresponde a la raíz cuadrada del MSE. Su principal ventaja es que conserva las mismas unidades de la variable objetivo, facilitando la interpretación del error:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (3)$$

Al igual que el MSE, el RMSE penaliza especialmente los errores de mayor magnitud. Por esta razón, valores elevados de RMSE pueden indicar la presencia de predicciones con errores considerables.

Finalmente, el **coeficiente de determinación (R^2)** permite medir qué proporción de la variabilidad de la variable objetivo es explicada por el modelo. Su expresión es:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4)$$

donde $\bar{y}$ representa el promedio de los valores reales. Un valor de R^2 cercano a 1 indica que el modelo explica una mayor proporción de la variabilidad observada en los datos. En términos generales, para las métricas MAE, MSE y RMSE se buscan valores bajos, mientras que para R^2 se buscan valores cercanos a 1.

5.2 Modelo - LSTM

Como primera alternativa de modelación se optó por una red neuronal recurrente tipo Long Short-Term Memory (LSTM). Es una arquitectura de redes neuronales recurrentes diseñada para trabajar con datos secuenciales y capturar dependencias temporales. Su principal característica es el uso de un estado de memoria y diferentes compuertas que permiten conservar, actualizar u olvidar información relevante a medida que procesa la secuencia. La elección responde principalmente a la naturaleza temporal del problema: al tratarse de una serie de tiempo, interesaba una arquitectura capaz de capturar las dependencias entre observaciones consecutivas. Para construir las entradas del modelo se trabajó con ventanas temporales: cada muestra agrupa un conjunto de observaciones históricas que se usan para predecir el valor de $PM_{2.5}$ en el instante siguiente. Se probó distintos tamaños de ventana y variaciones en la arquitectura hasta dar con una configuración que funcionara bien para el conjunto de datos.

Table 2: Resultados LSTM – Arquitectura: 1 Capa, 16 Neuronas

time_step	Épocas	MAE	MSE	RMSE	R^2
12	100	19,30	760,04	27,56	0,8787
24	100	10,56	312,05	17,66	0,9503
48	100	15,05	496,82	22,28	0,9210
168	100	23,16	1029,75	32,08	0,8359

Table 3: Resultados LSTM – Arquitectura: 2 Capas, 64 y 32 Neuronas

time_step	Épocas	MAE	MSE	RMSE	R^2
12	100	12,11	370,31	19,24	0,9409
24	100	12,23	352,76	18,78	0,9438
48	100	12,40	381,02	19,51	0,9394
168	100	11,77	336,89	18,35	0,9463

Tras comparar las distintas arquitecturas y configuraciones evaluadas, se decidió seleccionar el modelo LSTM de 1 capa con 16 neuronas y un time_step de 24, ya que presentó el mejor desempeño global entre todas las combinaciones probadas. Como se observa en la Tabla 2, esta configuración alcanzó un *MAE* de 10,56, un *MSE* de 312,05, un *RMSE* de 17,66 y un R^2 de 0.9503. Este resultado es muy importante porque muestra que una arquitectura más simple, con menos capas y neuronas, puede capturar la dinámica temporal de la variable $PM_{2.5}$ de manera adecuada. Esto se logra sin que el modelo se vuelva demasiado complicado ni requiera mucho poder computacional, como sucede con las redes más profundas.

Además, el tiempo de observación de 24 horas sugiere que un día completo de datos históricos es suficiente para predecir el valor siguiente. Si el tiempo de observación es más corto, como 12 horas, no se tiene suficiente información del pasado. Por otro lado, si el tiempo de observación es más largo, como 48 o 168 horas, se puede introducir ruido o información redundante que dificulta el aprendizaje del modelo.

Por lo tanto, se eligió esta configuración como modelo final de tipo LSTM, ya que ofrece un buen equilibrio entre precisión, simplicidad y eficiencia computacional. Para revisar el detalle de la construcción del modelo ir al notebook **02_LSTM_model**.

5.3 Modelo - XGBoost

Como segunda alternativa de modelación se utilizó XGBoost (Extreme Gradient Boosting), un algoritmo basado en árboles de decisión que construye el modelo de forma secuencial, donde cada nuevo árbol corrige los errores cometidos por los anteriores, mejorando progresivamente la capacidad predictiva del modelo. A diferencia de una regresión lineal tradicional, XGBoost es capaz de capturar relaciones no

lineales y de interacción entre las variables predictoras y la variable objetivo, lo que lo convierte en una alternativa interesante para comparar frente a la LSTM.

A diferencia de esta última, que procesa secuencias temporales de forma nativa, XGBoost recibe cada observación como un vector de características en formato tabular, sin ningún conocimiento implícito del orden temporal de los datos. Por esta razón, fue necesario un proceso de *feature engineering* para “traducir” la información temporal en variables explícitas que el modelo pudiera interpretar.

El paso más importante de este proceso fue la creación de variables de rezago (*lags*) a partir de la propia serie de PM2.5. La idea es simple: si queremos predecir el valor de contaminación en un instante t , es razonable pensar que los valores registrados en las horas inmediatamente anteriores ($t - 1$, $t - 2$, ..., $t - n$) contienen información valiosa sobre el comportamiento futuro de la variable.

Siguiendo esta lógica, se generaron 24 nuevas columnas en el dataset, una por cada hora de rezago, de la siguiente forma:

- *pm2.5_lag_1*: valor de PM2.5 registrado una hora antes.
- *pm2.5_lag_2*: valor de PM2.5 registrado dos horas antes.
- ⋮
- *pm2.5_lag_24*: valor de PM2.5 registrado 24 horas antes (es decir, el mismo momento del día anterior).

Se eligió una ventana de 24 horas debido a la naturaleza cíclica del fenómeno: la contaminación por PM2.5 suele estar asociada a patrones diarios (tráfico vehicular, actividad industrial, condiciones climáticas), por lo que incluir un día completo de historial permite que el modelo capture tanto la tendencia reciente como el comportamiento del día anterior a la misma hora. Además de los lags de PM2.5, se incorporaron al modelo las variables meteorológicas disponibles (temperatura, presión, humedad, velocidad del viento, entre otras) y la dirección del viento previamente codificada, con el objetivo de enriquecer el conjunto de características y brindarle al modelo contexto adicional sobre las condiciones ambientales asociadas a cada registro. Con este conjunto de variables –lags de PM2.5, variables meteorológicas y dirección del viento codificada– se conformó el dataset final utilizado para entrenar el modelo XGBoost.

Los resultados obtenidos por el modelo XGBoost se presentan en la Tabla 4. El modelo alcanzó un *MAE* de 10.5411, lo que indica que, en promedio, la diferencia absoluta entre la concentración de $PM_{2.5}$ real y la predicha es de aproximadamente 10.54 unidades de $PM_{2.5}$. El *RMSE* obtenido fue de 17.8105. Debido a que esta métrica penaliza en mayor medida los errores de mayor magnitud, su valor permite observar que existen algunas predicciones con errores superiores al error promedio representado por el *MAE*. La diferencia entre ambas métricas puede estar relacionada con la presencia de observaciones en las que el modelo presenta errores relativamente elevados. Por otra parte, el modelo obtuvo un coeficiente de determinación $R^2 = 0.9493$. Esto indica que, sobre el conjunto de prueba utilizado, el modelo logra explicar aproximadamente el 94.93% de la variabilidad observada en la concentración de $PM_{2.5}$. En conjunto, estos resultados muestran una buena capacidad predictiva del modelo bajo las condiciones experimentales establecidas.

Table 4: Resultados obtenidos por el modelo XGBoost

Métrica	Resultado
MAE	10.5533
MSE	317.4529
RMSE	17.8172
R^2	0.9493

Para un mayor detalle del modelo **XGBoost** implementado, ir al notebook **03_XGBoost_model**.

6 Selección del modelo final

Durante el proceso de modelación probamos distintas alternativas para predecir la concentración de PM2.5, entre ellas una arquitectura basada en redes neuronales recurrentes LSTM y el algoritmo XGBoost. A la hora de elegir el modelo final, no se tuvo en cuenta solo las métricas de desempeño; también

se tuvo en cuenta qué tan fácil era entrenarlo, qué tan interpretable resultaba, y qué tan práctico sería reutilizarlo más adelante.

La LSTM tuvo un desempeño competitivo, lo cual tiene sentido dado que es una arquitectura pensada justamente para problemas con dependencia temporal. Sin embargo, su implementación no fue tan sencilla: Se tuvo que construir secuencias de capas neuronales (prueba y error), definir ventanas temporales y ajustar varios hiperparámetros propios de la arquitectura de la red, lo que la vuelve un modelo más costoso de entrenar y experimentar. XGBoost, en cambio, permitió trabajar directamente con las características tabulares que ya se habían construido a partir del dataset —variables meteorológicas, dirección del viento y los valores históricos de PM2.5 a través de variables *lag*—. Esto simplificó bastante el proceso de entrenamiento y dio más margen para experimentar con distintas configuraciones sin tanta complicación.

Adicionalmente, XGBoost ofrece una ventaja importante en cuanto a usabilidad: una vez entrenado, el modelo puede guardarse y reutilizarse más adelante para hacer predicciones sobre datos nuevos, sin necesidad de volver a pasar por todo el proceso de entrenamiento. Esto resulta especialmente útil de cara a integrar el modelo en un flujo de predicción real, donde conviene mantener separadas las etapas de entrenamiento e inferencia.

Considerando el conjunto de configuraciones evaluadas, XGBoost se seleccionó como modelo final, ya que no solo obtuvo el mejor desempeño en conjunto, sino que además su implementación resultó más simple y directa para el problema abordado.